DFS

Materia	Algoritmi
Anno	Secondo Anno
Data	@December 20, 2024
Studiato	Done

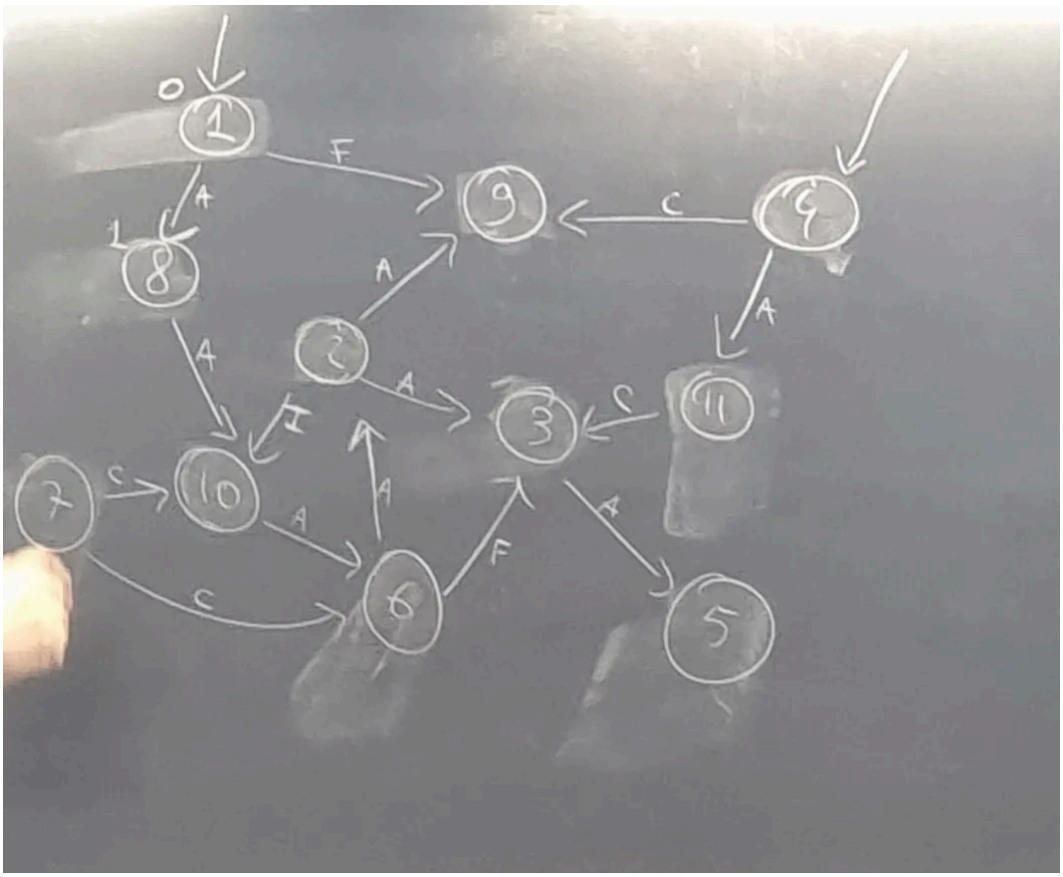
La **ricerca in profondità** di un grafo, o **Depth First Search (DFS)**, esplora gli archi che dipartono dal vertice v scoperto più recentemente e che sono ancora inesplorati. Una volta che tutti gli archi uscenti da v sono stati esplorati, la ricerca fa retromarcia.

Il processo continua finché tutti i vertici raggiungibili dal vertice sorgente sono stati esplorati. Se nel grafo ci sono ancora dei vertici non esplorati, la ricerca DFS seleziona un nuovo vertice sorgente e ripete la procedura.

Si crea un grafo dei predecessori, ossia una foresta che contiene uno o più alberi DF. In particolare, si avrà $G_\pi = (V, E_\pi)$ con $E_\pi = \{ (v.\pi, v) : v \in V \land v.\pi \neq \text{null} \}$, dove $v.\pi$ è il predecessore di v nell'esplorazione DFS.

A differenza della BFS, la scelta della sorgente ha meno importanza per l'applicazione della DFS. Per questo motivo si utilizza un **ordinamento lessicografico** dei nodi del grafo, quindi se i vertici sono numerati a partire da 1 si parte proprio da 1.

Nella DFS si esegue una visita in profondità, ossia ci spostiamo subito in uno dei nodi adiacenti tralasciando i restanti nel tentativo di andare più in profondità possibile allontanandomi dal nodo 1.
Supponiamo il grafo:



e iniziamo da 1. Da uno passiamo a 8 tralasciando 9, poi ci spostiamo verso il 10 e poi verso il 6. Ogni nodo visitato viene colorato di grigio. Dal 6 passiamo al 2, dal 2 passiamo al 3 e dal 3 al 5. Qui si ha un **vicolo cieco**: non possiamo attraversare alcun arco. Tale nodo allora viene etichettato di nero e si torna indietro utilizzando i nodi grigi.
Se tornando indietro si osservano noti adiacenti non esplorati, allora visitiamo tali nodi. Dal
5 si torna al 3, altro vicolo cieco che diventa nero, poi al 2, che permette di spostarci al 9, vicolo cieco che diventa nero. Si procede allo stesso modo per ogni vertice visitabile.

Non abbiamo tuttavia visitato tutto il grafo. Se ci sono altri nodi bianchi, si sceglie il nodo bianco più piccolo e si esplorano i nodi non visitati allo stesso modo. Allora si ha che un nodo è **bianco** se il nodo non è visitato, **grigio** se il nodo viene visitato e il ramo è in esplorazione, **nero** se il nodo non ha percorsi esplorabili.

La visita DFS, inoltre, genera una **foresta**, in cui ogni albero rappresenta i cammini a partire dai vertici sorgenti scelti nella procedura DFS. La foresta è un sottografo

in cui non sono presenti tutti gli archi. Oltre agli archi nella foresta distinguiamo:

- gli **archi in avanti**, che uniscono un nodo di un albero con un suo discendente non diretto all'interno dell'albero;
- gli **archi di attraversamento**, che uniscono rami diversi dello stesso albero oppure rami diversi di alberi diversi;
- gli **archi all'indietro**, che unisce un nodo con un suo predecessore all'interno dell'albero.

Un arco che collega un nodo grigio a un nodo bianco durante la visita in profondità del grafo è presente all'interno della foresta DFS.
Se esiste un arco che collega due nodi grigi durante la visita di un grafo, tale arco è un arco all'indietro. Allora se esiste tale arco, possiamo dire che **il grafo presenta un ciclo**.

Se esiste un arco che collega un nodo grigio ad un nodo nero, tale arco potrebbe essere di attraversamento o in avanti. Tuttavia, **non abbiamo sufficienti informazioni per determinare ciò**. Abbiamo allora bisogno di un ulteriore dato.

Attraversiamo il grafo e stavolta teniamo conto del tempo di esplorazione del grafo. Quindi al tempo 0 esploriamo 1, al tempo 1 esploriamo 8, e poi si va avanti con le coppie tempo-nodo $(2, 10), (3, 6), (4, 2), (5, 3), (6, 5)$. Ora che non abbiamo alcun nodo da esplorare, segniamo un'altro dato tempistico, ossia il tempo di conclusione della visita, ossia il momento in cui il nodo diventa nero. Per ogni nodo, quindi, teniamo traccia del tempo di inizio e di fine visita. Se teniamo conto dell'intervallo $[t.inizio, t.fine]$ (i tempi saranno indicati con d, "discovery", e f, "finish") e consideriamo le coppie di nodi (u, v), abbiamo due casi:

1. l'intervallo di v è incluso interamente nell'intervallo di u;
2. l'intervallo di v è disgiunto dall'intervallo di u.

Non esiste il caso in cui l'intervallo di u e l'intervallo di v si intersecano ma non sono uno incluso nell'altro. Questo in virtù della natura della definizione delle etichette temporali.

Se ci troviamo nella situazione 1, allora v è discendente di u. Altrimento, sono su alberi diversi.

Questo ci permette di capire se abbiamo un arco in avanti o un arco di attraversamento. L'arco (u, v) è un arco in avanti se u è incluso in v, altrimenti è un arco di attraversamento. In simboli:

$$u.d > v.d \implies \text{arco di attraversamento}$$

Analogamente, diremo che:

$$u.d < v.d \implies \text{arco in avanti}$$

Scriviamo allora la procedura.

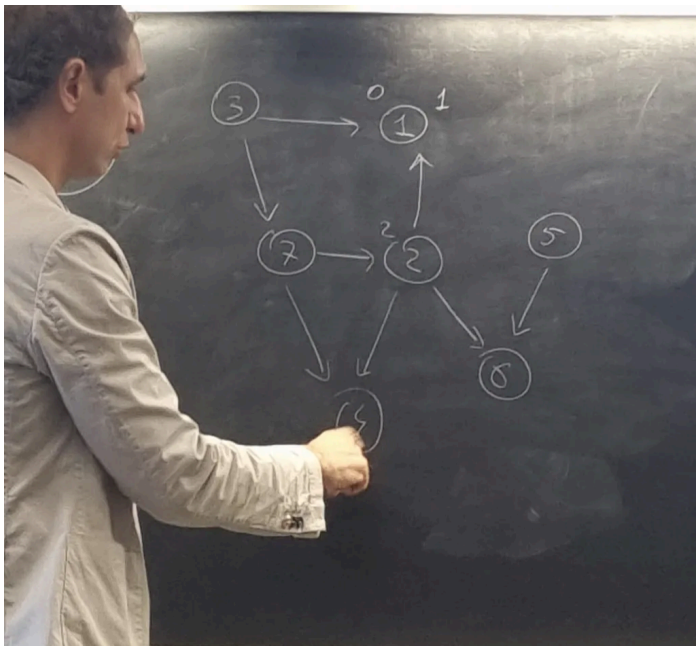
```
DFS(V):
   for each v in V do
      color[v] = white
      pi[v] = null // pi = predecessore
      d[v] = f[v] = +infin
   t = 0    // immaginata come variabile globale
   for each v in V do
      if color[v] = white then
         DFS-Visit(v)

DFS-Visit(v):
   color[v] = gray
   d[v] = t
   t = t + 1
   for each u in adj[v] do
      if color[u] = white then
         pi[u] = v
         DFS-Visit(u)
   color[v] = black
   f[v] = t
   t = t + 1
```

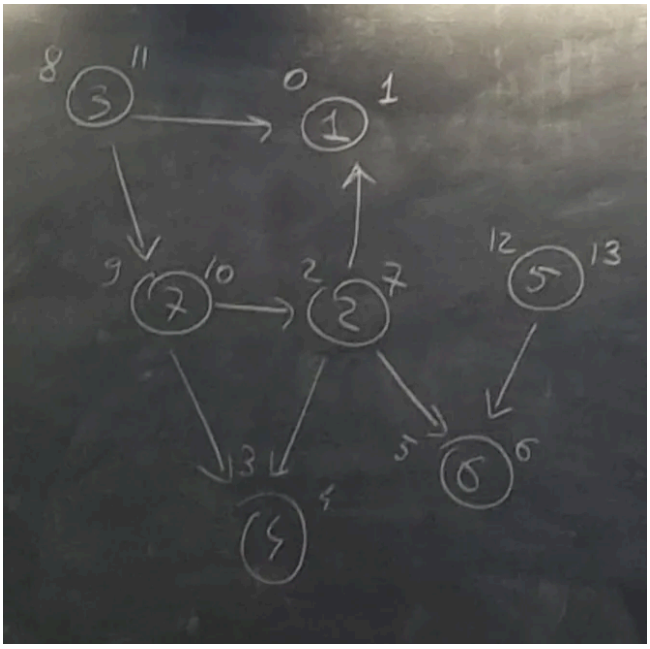
Per verificare che il grafo è aciclico, modifichiamo la procedura `DFS-Visit` e controlliamo anche se `color[u] = gray`.

Ordinamento topologico

Possiamo utilizzare la visita DFS per ottenere un ordinamento topologico. Consideriamo il grafo:



ed eseguiamo la visita DFS. Otteniamo:

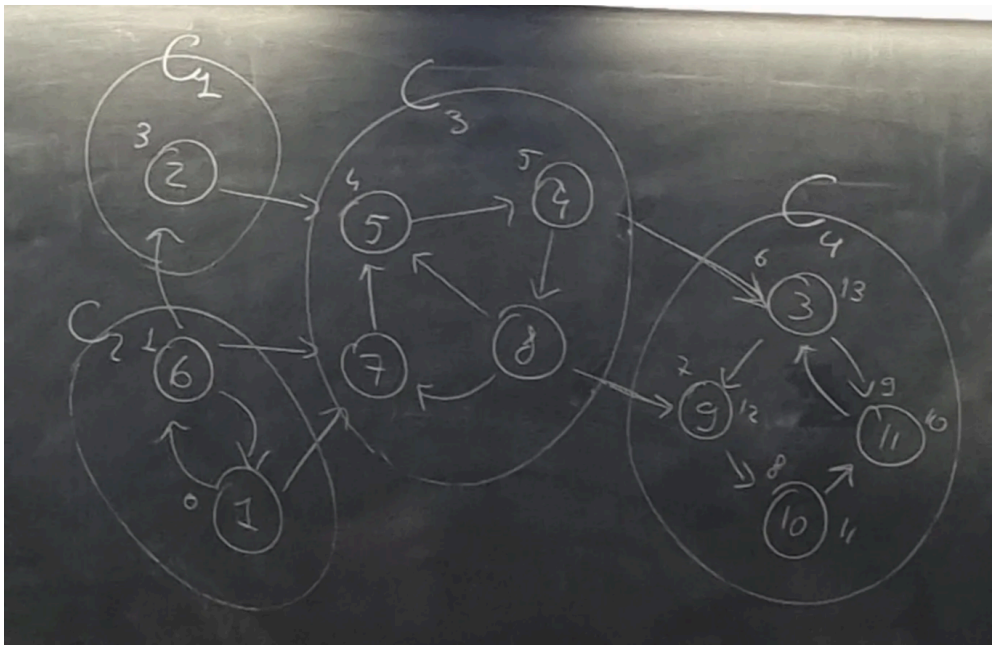


Osserviamo che se $d.u < d.v$ nell'arco (u, v) allora u è un predecessore di v ed esiste un cammino da u a v. Ciò vale anche nel caso in cui $f.u > f.v$. Tra i due tuttavia conviene utilizzare i tempi di fine visita. Quindi se $f.u > f.v$, allora nell'ordinamento topologico vale $u < v$.

Per creare l'ordinamento topologico, allora, basta che eseguiamo la DFS, e ogni volta che marchiamo di nero un nodo lo inseriamo in una stack inizialmente vuota. L'ordine topologico si ottiene estraendo gli elementi dalla stack.

Componenti fortemente connesse

Sia dato il seguente grafo di componenti fortemente connesse C_1, C_2, C_3, C_4:



Partire da 1 non è una scelta furba, in quanto abbiamo ottenuto una foresta che unisce tutte le componenti fortemente connesse.
Infatti si ha che la struttura delle componenti fortemente è la seguente:

graph LR
C1 → C3
C3 → C4
C2 → C1
C2 → C3

Quindi conviene partire dalla componente C_4.
Fatta questa considerazione, ordiniamo i nodi per tempo di fine visita. Possiamo ottenere un
ordinamento topologico del grafo dei componenti soprastante, in quanto è aciclico e ammette un ordinamento topologico.

Per calcolare il grafo delle componenti fortemente connesse si fa la seguente:

- si applica la DFS;
- **seguendo l'ordinamento decrescente secondo l'etichetta di fine visita,** applichiamo una seconda DFS al grafo che si ottiene **invertendo la direzione degli archi**, ossia il **grafo trasposto** G^T. Le componenti fortemente connesse sono preservate;
- l'insieme degli alberi della foresta della seconda DFS è il grafo delle componenti fortemente connesse.

La procedura è sempre $O(|V| + |E|)$.